2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C) | 979-8-3503-5157-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/SANER-C62648.2024.00025

The Quantum Circuit Model is not a Practical Representation of Quantum Software

Arianne Meijer–van de Griend
University of Helsinki
Helsinki, Finland
ariannemeijer@gmail.com

***Abstract*—This position paper argues that the quantum circuit model is not a practical representation of quantum software for three reasons. First, the quantum circuit model is not suitable for semantical reasoning and better mathematical frameworks exist. Second, the abstraction level is too low for programming and too high for pulse-level machine code. Lastly, by ignoring the existence of a target quantum computer on which the quantum software could run, the transformation from software to quantum circuit model makes assumptions that result in inefficient machine code lower in the quantum computing stack.**

Thus, we conclude that new interfaces between quantum software, compiler, and quantum hardware need to be created such that the abstraction level is both suitable for the software and the hardware.

I. Introduction

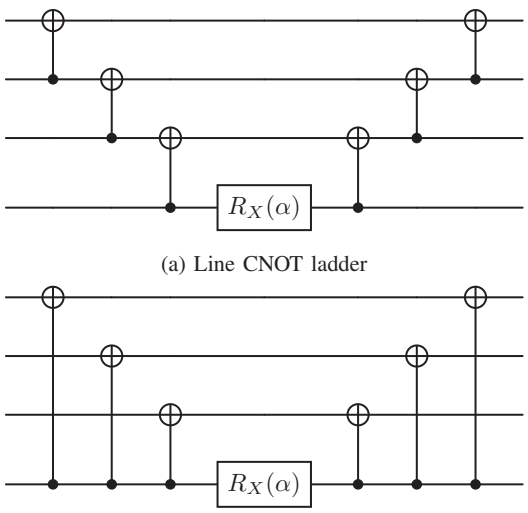
With the recent advances in quantum hardware [1] comes a need for appropriate tools for the analysis, evolution, and reengineering of quantum software. At the time of writing, one of the primary tools for representing quantum software is the quantum circuit model [22]. This model represents the operations of a quantum program in a diagrammatic style showing the order of operations with respect to time and the qubits on which these operations are performed.

The quantum circuit model, first introduced by Richard Feynman in 1986 [12], has been a valuable tool in the past as a computational construct that could be used for reasoning about what quantum software might look like. However, now that the first quantum computers are available to the public online [1] and with the advances in classical programming since 1986, it becomes more clear that the universality of the quantum circuit model is not a sufficient argument for its use in practice.

A practical representation of quantum software needs to allow reasoning about the quantum computations. This reasoning needs to be done at a suitable abstraction level that represents the use of the quantum software. Moreover, this needs to be done in a way that is suitable for current and future quantum devices. In this short position paper, we will argue that the quantum circuit model is unable to do any of these three things.

II. Semantical Reasoning

First, we discuss the suitability of the quantum circuit model for reasoning about its computational meaning (i.e. semantics). Because quantum simulation is infeasible, we need a representation of the quantum software that allows abstract reasoning about quantum software without ever calculating the state of the qubits.



(a) Line CNOT ladder

(b) Star CNOT ladder

Fig. 1: Two equivalent example circuits implementing $e^{-i\frac{\alpha}{2}XXXX}$.

However, the quantum circuit notation can sometimes be misleading. The representation of a CNOT makes it seem like the control qubit remains unchanged, but this is not always the case. In fact, it is an important property that it does change for amplitude amplification [4, 15]. For example, the CNOTs before the X rotation in Figure 1 cannot be optimized away.

These classical conventions in notations makes the user think about the quantum circuit in a classical way that does not reflect what the quantum circuit does. Additionally, the notation does not easily show which operations commute with each other and which do not.

These lacking properties of the quantum circuit model make it difficult to determine equalities between different quantum circuits (see e.g. figure 1). This makes the analysis and optimization of quantum software from its quantum circuit representation very difficult and sometimes even counter-productive [8]. In practice, these equalities are proven by

979-8-3503-5157-6/24/$31.00 ©2024 IEEE
DOI 10.1109/SANER-C62648.2024.00025

Authorized licensed use limited to: Universitas Airlangga. Downloaded on July 24,2026 at 07:19:12 UTC from IEEE Xplore. Restrictions apply.

translating the quantum circuit into a different formalism, e.g. tensor networks [23], path-sums [3], ZX-calculus [6], stabilizer decompositions [5], or plain mathematics. Thus, better formalisms for reasoning exist. In some occasions, these have even been used to find a bug in a previous algorithm [16, 21].

III. Abstraction Level

The quantum circuit notation follows the old classical circuit representation that is no longer used for representing classical programs. Even classically, thinking about software on the level of binary logic gates is no longer suitable. Similarly, current quantum software is not written in the abstraction level of quantum logic gates. In practice, quantum software is written using larger data structures that are handled by an API, like a Hamiltonian or an entangling layer of a particular type. As such, the API-style programming of quantum computers resembles the programming of neural networks much more closely than assembly-level programming.

Moreover, the quantum circuit model is not suitable for describing hybrid quantum-classical optimization algorithms, such as VQE [11] and QAOA [10], in a holistic fashion. Generally, the quantum circuit is only used to describe the small part of the algorithm that is executed on the quantum computer, without describing the classical feedback loop surrounding it. This obscures how the quantum algorithm and classical algorithm connect.

Additionally, the quantum circuit representation generally assumes a Clifford+R_Z gate set. Although this is a universal gate set that the quantum computers can implement, it is not necessarily representative of machine-level operations that the quantum computers can perform [9]. Quantum computers perform pulses implementing operations that are often in the form $e^{-i\alpha\otimes_i P_i}$, where P_i is a Pauli operation. These operations are then calibrated to form a Clifford+R_Z gate set, but that is not strictly necessary [20].

Thus, the abstraction level of a quantum circuit is not suitable for programming quantum software, nor for the execution of a quantum program on a quantum computer. Next, we will argue that it is not a good intermediate representation for quantum compilation either.

IV. Targeting Devices

Although the quantum circuit model is universal for quantum computing, it assumes that the hardware implements a two-qubit gate as its only entangling gate (e.g. a CNOT or CZ gate) and that they can be performed between arbitrary qubits on the device. This is generally not true. Many quantum technologies (e.g. superconducting qubits) have restrictions regarding between which qubits their multi-qubit operation can be performed and some technologies (e.g. ion traps) allow global multi-qubit operations over all qubits.

When quantum circuits are generated using these assumptions, it is easy to make inefficient choices about which gates to use where (see e.g. figure 1) which results in the qubit routing problem [24, 7]. Combined with the difficulties of software analysis with quantum circuits described before, this results in inefficient machine code when compiling the circuit to a target device. Even when taking a target quantum computer into account when designing a quantum circuit, this makes it difficult to reuse it for another computer. Moreover, compiling the quantum circuit for a different gate set or qubit connectivity is difficult and inefficient [7] because of the abstraction level [17].

Particularly for the small noisy quantum hardware of the NISQ era, these inefficiencies could make the difference between running an algorithm now or in a few years. This impacts the timeline for quantum advantage greatly.

Because of the separation of concerns in the stack, it should not be expected that programmers have a deep understanding of the different target devices. This is not expected from classical programming; very few programmers know about branch mispredictions, SIMD, or CUDA architectures. Neither should we expect quantum programmers to write efficient low-level software. This should be handled by an efficient compiler that can transform the high-level instructions into the gate set that the quantum computer actually uses.

However, it has been shown that quantum compilers benefit from using larger computational structures, e.g. Paulistrings ($e^{-i\alpha\otimes_i P_i}$), than the Clifford+$R_Z$ gate set [25, 2, 13, 14, 18, 19, 17]. The combination of the difficulty of semantical reasoning and the level of abstraction in the quantum circuit model makes it that the compiler can only make inefficient local adjustments to the circuit. If the quantum program needs to be transformed into an intermediate representation, it does not matter if the input and output is a quantum circuit or some other representation. So the compiler does not benefit from the quantum circuit model.

V. Conclusion

In conclusion, we need to step away from the quantum circuit model in the same vein as we have stepped away from programming in assembly. When analyzing quantum programs, either for debugging, optimization, compilation, or understanding, the quantum circuit model does not help us. The quantum circuit model uses an abstraction level that is not compatible with the use cases of quantum software.

Thus, the quantum circuit model is only used as an abstraction level that serves as an interface between users working on different layers of the quantum stack. The existence of a common interface does not imply its suitability. Moreover, more naturally suitable interfaces are available: the software can be written using existing toolkits, the hardware can be specified using the control hardware API, and the compiler can use its intermediate representation to translate between them.

The separation of responsibilities does not rule out the need for communication. New formalisms are needed to define the interfaces between layers of the quantum stack such that they can be standardized. As are result, we will be able to make more efficient use of the quantum resources available today.

Authorized licensed use limited to: Universitas Airlangga. Downloaded on July 24,2026 at 07:19:12 UTC from IEEE Xplore. Restrictions apply.

References

[1] A-tA-v et al. *Qiskit: An Open-source Framework for Quantum Computing*. 2021. DOI: 10.5281/zenodo.2573505.

[2] Matthew Amy and Vlad Gheorghiu. "Staq —A Full-Stack Quantum Processing Toolkit". In: *Quantum Science and Technology* 5.3 (June 2020), p. 034016. ISSN: 2058-9565. DOI: 10.1088/2058-9565/ab9359.

[3] Matthew Amy, Dmitri Maslov, and Michele Mosca. "Polynomial-Time T-Depth Optimization of Clifford+T Circuits Via Matroid Partitioning". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33.10 (Oct. 2014), pp. 1476–1489. ISSN: 1937-4151. DOI: 10.1109/TCAD.2014.2341953.

[4] G. Brassard and P. Hoyer. "An exact quantum polynomial-time algorithm for Simon's problem". In: *Proceedings of the Fifth Israeli Symposium on Theory of Computing and Systems*. 1997, pp. 12–23. DOI: 10.1109/ISTCS.1997.595153.

[5] Sergey Bravyi et al. "Simulation of quantum circuits by low-rank stabilizer decompositions". In: *Quantum* 3 (Sept. 2019), p. 181. ISSN: 2521-327X. DOI: 10.22331/q-2019-09-02-181.

[6] Bob Coecke and Ross Duncan. "Interacting Quantum Observables". In: *Automata, Languages and Programming*. Ed. by Luca Aceto et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 298–310. ISBN: 978-3-540-70583-3.

[7] Alexander Cowtan et al. "On the Qubit Routing Problem". In: (2019), 32 pages. DOI: 10.4230/LIPICS.TQC.2019.5.

[8] Alexander Cowtan et al. "Phase Gadget Synthesis for Shallow Circuits". In: *Electronic Proceedings in Theoretical Computer Science* 318 (May 2020), pp. 213–228. ISSN: 2075-2180. DOI: 10.4204/EPTCS.318.13.

[9] Nathan Earnest, Caroline Tornow, and Daniel J. Egger. "Pulse-Efficient Circuit Transpilation for Quantum Applications on Cross-Resonance-Based Hardware". In: *Physical Review Research* 3.4 (Oct. 2021), p. 043088. ISSN: 2643-1564. DOI: 10.1103/PhysRevResearch.3.043088.

[10] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. "A quantum approximate optimization algorithm". In: *arXiv preprint arXiv:1411.4028* (2014). DOI: 10.48550/arXiv.1411.4028.

[11] Dmitry A Fedorov et al. "VQE method: a short survey and recent developments". In: *Materials Theory* 6.1 (2022), pp. 1–21. DOI: 10.1186/s41313-021-00032-6.

[12] Richard P Feynman. "Quantum mechanical computers." In: *Found. Phys.* 16.6 (1986), pp. 507–532. DOI: 10.1007/BF01886518.

[13] Vlad Gheorghiu et al. "Reducing the CNOT Count for Clifford+T Circuits on NISQ Architectures". In: (2020). DOI: 10.48550/ARXIV.2011.12191.

[14] Stefano Gogioso and Richie Yeung. "Annealing Optimisation of Mixed ZX Phase Circuits". In: (2022). DOI: 10.48550/ARXIV.2206.11839.

[15] Lov K. Grover. "Quantum Computers Can Search Rapidly by Using Almost Any Transformation". In: *Phys. Rev. Lett.* 80 (19 May 1998), pp. 4329–4332. DOI: 10.1103/PhysRevLett.80.4329.

[16] Aleks Kissinger and John van de Wetering. "Reducing the number of non-Clifford gates in quantum circuits". In: *Phys. Rev. A* 102 (2 Aug. 2020), p. 022406. DOI: 10.1103/PhysRevA.102.022406.

[17] Gushu Li et al. "Paulihedral: A Generalized Block-Wise Compiler Optimization Framework for Quantum Simulation Kernels". In: *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Feb. 2022, pp. 554–569. DOI: 10.1145/3503222.3507715.

[18] Simon Martiel and Timothée Goubault de Brugière. "Architecture Aware Compilation of Quantum Circuits via Lazy Synthesis". In: *Quantum* 6 (June 2022), p. 729. ISSN: 2521-327X. DOI: 10.22331/q-2022-06-07-729.

[19] Adriana Meijer - van de Griend and Ross Duncan. "Architecture-Aware Synthesis of Phase Polynomials for NISQ Devices". In: (2020). DOI: 10.48550/ARXIV.2004.06052.

[20] André Melo, Nathan Earnest-Noble, and Francesco Tacchino. "Pulse-efficient quantum machine learning". In: *Quantum* 7 (Oct. 2023), p. 1130. ISSN: 2521-327X. DOI: 10.22331/q-2023-10-09-1130.

[21] Yunseong Nam et al. "Automated optimization of large quantum circuits with continuous parameters". In: *npj Quantum Information* 4.1 (2018), p. 23. DOI: 10.1038/s41534-018-0072-4.

[22] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th anniversary ed. Cambridge ; New York: Cambridge University Press, 2010. ISBN: 978-1-107-00217-3.

[23] Román Orús. "Tensor networks for complex quantum systems". In: *Nature Reviews Physics* 1.9 (2019), pp. 538–550. DOI: 10.1038/s42254-019-0086-7.

[24] Alexandru Paler. "On the Influence of Initial Qubit Placement During NISQ Circuit Compilation". In: (2018). DOI: 10.48550/ARXIV.1811.08985.

[25] P V Sriluckshmy et al. "Optimal, hardware native decomposition of parameterized multi-qubit Pauli gates". In: *Quantum Science and Technology* 8.4 (Sept. 2023), p. 045029. DOI: 10.1088/2058-9565/acfa20.

Authorized licensed use limited to: Universitas Airlangga. Downloaded on July 24,2026 at 07:19:12 UTC from IEEE Xplore. Restrictions apply.